

SIMATS ENGINEERING
CO1-ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1103

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I GIRIJA B / 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

Short Answer Test

1. Smart Banking System

(a) Object-Oriented Concepts

1. Class

A **class** is a blueprint or template used to create objects. It defines the attributes and methods of an object.

Example:

A BankAccount class may contain:

- Account number
- Account holder name
- Balance

Methods:

- deposit()
- withdraw()
- transferFunds()

The class defines what a bank account can contain and do.

2. Object

An **object** is an instance of a class. It represents a real-world entity in the software system.

Example:

If BankAccount is a class, then an account belonging to a particular customer is an object.

```
BankAccount account1 = new BankAccount();
```

Here, account1 is an object of the BankAccount class.

3. Message Passing

Message passing is the process by which objects communicate with each other by calling methods.

Example:

A Customer object may send a message to a BankAccount object:

```
account.transferFunds(amount);
```

The customer requests the bank account object to perform a fund transfer.

4. Encapsulation

Encapsulation means combining data and methods into a single class and restricting direct access to internal data.

For example, the bank balance should not be directly modified by users.

private balance;

The balance can be changed only through methods such as:

deposit();

withdraw();

This protects sensitive banking data.

5. Polymorphism

Polymorphism means "many forms." It allows the same method or interface to perform different operations depending on the object.

Example:

makePayment();

This method may be implemented differently for:

- Credit card payment
- Debit card payment
- UPI payment
- Net banking payment

Thus, the same operation can have different implementations.

(b) Object Relationships

1. Association

Association represents a general relationship between two independent objects.

Example:

Customer ----- BankAccount

A customer has a bank account. Both the Customer and BankAccount objects can exist independently.

2. Aggregation

Aggregation represents a **weak whole-part relationship**.

Example:

Bank ----- Customers

A bank contains many customers. However, if the bank object is deleted, the customer objects can still exist independently.

3. Composition

Composition represents a **strong whole-part relationship**. The part cannot exist independently of the whole.

Example:

BankAccount ----- AccountTransaction

If a bank account is permanently deleted, its transaction records may also be removed as part of that account's composition.

(c) Benefits of These Concepts

These object-oriented concepts improve **reusability and maintainability** in a banking system.

- **Classes** allow common functionality to be reused.
- **Objects** represent real-world banking entities clearly.
- **Encapsulation** protects sensitive information such as passwords and account balances.
- **Polymorphism** allows different payment methods to be added without changing existing code.
- **Inheritance and reusable classes** reduce code duplication.
- **Association, aggregation, and composition** clearly define relationships between banking objects.

For example, if a new payment method such as digital wallet payment is added, a new payment class can be created without modifying the entire banking system. Therefore, object-oriented design makes the system easier to extend, test, maintain, and reuse.

2. Smart Healthcare Management System

(a) Object-Oriented Design Principles

1. Abstraction

Abstraction means showing only essential features and hiding unnecessary implementation details.

Example:

A doctor uses:

```
bookAppointment()
```

The doctor does not need to know the internal database operations used to store the appointment.

Abstraction reduces complexity and allows users to focus on important operations.

2. Encapsulation

Encapsulation combines data and methods inside a class and protects data from unauthorized access.

For example, patient medical information can be declared as private:

```
private medicalRecords;
```

The data can be accessed only through authorized methods such as:

```
getMedicalRecord();
```

```
updateMedicalRecord();
```

This improves data security and privacy.

3. Modularity

Modularity means dividing a large system into smaller independent modules.

A healthcare system can contain:

- Patient Management Module
- Doctor Management Module
- Appointment Module
- Billing Module
- Medical Records Module

Each module performs a specific function. This makes the system easier to develop, test, and maintain.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

The **Liskov Substitution Principle** states that objects of a subclass should be replaceable for objects of the superclass without affecting the correctness of the program.

Example:

Suppose:

Doctor

is a superclass and:

GeneralDoctor

SpecialistDoctor

are subclasses.

Both subclasses should be usable wherever a Doctor object is expected. They should correctly perform the common operations of the Doctor class.

This prevents unexpected behavior and improves software reliability.

2. Interface Segregation Principle (ISP)

The **Interface Segregation Principle** states that a class should not be forced to implement methods that it does not use.

Instead of creating one large interface:

HealthcareOperations

with many unrelated methods, separate interfaces can be created:

AppointmentService

BillingService

MedicalRecordService

A class can implement only the interface that it actually requires.

(c) Benefits to Software Quality and Flexibility

These principles improve software quality by:

- Reducing unnecessary complexity through abstraction.
- Protecting sensitive patient data through encapsulation.
- Dividing the system into manageable modules.

- Improving flexibility through LSP.
- Preventing unnecessary dependencies through ISP.
- Making the system easier to test and modify.
- Allowing new healthcare services to be added without changing existing modules.

For example, a new online consultation feature can be added as a separate module without affecting the existing appointment and billing systems.

3. Online Food Delivery System

(a) Phases of SDLC

1. Planning

Planning is the first phase of the SDLC.

In an online food delivery system, the development team decides:

- Project objectives
- Project scope
- Required resources
- Estimated time
- Estimated cost
- Potential risks

Purpose:

To determine whether the project is feasible and create a proper development plan.

2. Requirement Analysis

In this phase, the requirements of users and stakeholders are collected and analyzed.

For an online food delivery system, requirements may include:

- Customer registration
- Restaurant listing
- Food search
- Online ordering
- Payment

- Order tracking
- Delivery management

Purpose:

To clearly understand what the system must do.

3. Design

The system architecture and detailed design are created.

The design may include:

- Database design
- User interface design
- Class diagrams
- Component diagrams
- System architecture
- Security design

Purpose:

To provide a blueprint for developing the system.

4. Development

The actual coding of the software is performed.

Developers implement:

- Customer module
- Restaurant module
- Order module
- Payment module
- Delivery tracking module

Purpose:

To convert the design and requirements into a working software system.

5. Testing

The developed software is tested to find and correct errors.

Testing may include:

- Unit testing
- Integration testing
- System testing
- User acceptance testing

For example, the team checks whether:

Customer → Selects Food → Places Order → Makes Payment
works correctly.

Purpose:

To ensure that the software is reliable and meets user requirements.

6. Maintenance

After deployment, the software is maintained and improved.

Maintenance includes:

- Fixing bugs
- Improving performance
- Adding new features
- Updating security
- Supporting new devices

Purpose:

To keep the software useful and functional after deployment.

(b) Purpose of Each Phase

SDLC Phase	Main Purpose
Planning	Decide project feasibility and scope
Requirement Analysis	Understand user needs
Design	Create the system blueprint

SDLC Phase	Main Purpose
Development	Build the software
Testing	Find and fix errors
Maintenance	Improve and support the system

(c) How SDLC Delivers Reliable Software

SDLC provides a systematic and organized approach to software development.

It improves reliability by:

- Clearly identifying requirements.
- Planning resources and risks.
- Designing the system before coding.
- Testing the software thoroughly.
- Fixing errors before deployment.
- Continuously maintaining the software.

For example, in a food delivery system, testing ensures that payment, order processing, and delivery tracking work correctly. Therefore, SDLC reduces development errors and helps deliver high-quality and reliable software.

4. Smart Agriculture Monitoring System

(a) SDLC Models

1. Prototype Model

The **Prototype Model** creates an early working model of the system.

In a smart agriculture system, a prototype may include:

- Soil moisture monitoring
- Temperature display
- Basic irrigation control

Users can review the prototype and provide feedback.

Advantages:

- Helps clarify unclear requirements.
 - Allows early user feedback.
 - Reduces misunderstanding of requirements.
-

2. Spiral Model

The **Spiral Model** combines iterative development with risk analysis.

Each cycle includes:

1. Planning
2. Risk analysis
3. Development
4. Evaluation

For example, risks such as sensor failure, inaccurate data, and network problems can be identified and addressed during development.

Advantages:

- Strong risk management.
 - Suitable for complex projects.
 - Supports repeated improvements.
-

3. Agile Model

The **Agile Model** develops software in small iterations called sprints.

For a smart agriculture system:

- Sprint 1: Soil monitoring
- Sprint 2: Weather monitoring
- Sprint 3: Automatic irrigation
- Sprint 4: Mobile alerts

Customer feedback is continuously collected.

Advantages:

- Easily handles changing requirements.

- Provides frequent working software.
 - Encourages customer collaboration.
-

4. DevOps Model

The **DevOps Model** combines development and operations.

It focuses on:

- Continuous Integration
- Continuous Delivery
- Automated Testing
- Continuous Deployment
- Monitoring

For example, updates to an agriculture monitoring application can be automatically tested and deployed.

Advantages:

- Faster software delivery.
 - Automation.
 - Continuous monitoring.
 - Faster bug fixing.
-

(b) Prototype Model vs Agile Model

Feature	Prototype Model	Agile Model
Main Focus	Understanding requirements	Continuous development
Development	Creates an early model	Develops working increments
User Feedback	Mainly during prototype stage	Continuous throughout development
Requirements	Helps clarify unclear requirements	Can change during every iteration
Delivery	Prototype may not be final software	Regular working software is delivered
Best Used For	Unclear requirements	Frequently changing requirements

(c) Recommended SDLC Model

The **Agile Model** is the most suitable model for the Smart Agriculture Monitoring System.

Justification:

- Customer requirements change regularly.
- New sensors and technologies may need to be added.
- The system can be developed in small modules.
- Farmers can provide continuous feedback.
- New features can be prioritized based on actual needs.
- Working software can be delivered frequently.

For example, the team can first develop soil moisture monitoring and later add weather prediction, automatic irrigation, and mobile notifications.

Therefore, Agile provides the flexibility required for this project.

5. Smart Transport Management System

(a) Importance of UML

UML (Unified Modeling Language) is a standard visual modeling language used to analyze, design, and document software systems.

UML is important because it:

- Represents system structure visually.
- Shows relationships between objects and components.
- Helps developers understand system requirements.
- Supports object-oriented analysis and design.
- Reduces misunderstandings.
- Provides documentation for future maintenance.

For a smart transport system, UML can represent vehicles, passengers, drivers, routes, GPS services, and payment systems.

(b) UML Diagrams

1. State Diagram

A **State Diagram** shows the different states of an object and the events that cause transitions between states.

Example: Transport Booking

Requested → Confirmed → Driver Assigned → In Transit → Completed

If the customer cancels the booking:

Requested → Cancelled

It is useful for systems where objects change states over time.

2. Component Diagram

A **Component Diagram** shows the major software components and their dependencies.

For a smart transport system, components may include:

- User Application
- GPS Service
- Booking Service
- Payment Service
- Notification Service
- Database

It shows how different software components interact with each other.

3. Deployment Diagram

A **Deployment Diagram** shows the physical arrangement of hardware and software.

Example:

Mobile Device

|

↓

Application Server

|



Database Server

It may also show:

- GPS devices
- Cloud servers
- IoT sensors
- Mobile devices

It explains where software components are deployed.

4. Package Diagram

A **Package Diagram** organizes related classes and components into packages.

For example:

User Management

└─ Passenger

└─ Driver

Transport Management

└─ Vehicle

└─ Route

Payment Management

└─ Payment

└─ Transaction

It helps organize a large system into logical groups.

(c) UML and Stakeholder Communication

UML models improve communication among:

- Customers

- Business analysts
- Designers
- Developers
- Testers
- Project managers

Different stakeholders may have different technical knowledge. UML diagrams provide a common visual language.

For example:

- A customer can understand a use case or activity diagram.
- A developer can understand a class or component diagram.
- A system administrator can understand a deployment diagram.

Therefore, UML reduces ambiguity, improves collaboration, and ensures that everyone has a common understanding of the system.

6. Smart Inventory Management System

(a) Modular Software Design

Modular software design is the process of dividing a large software system into smaller, independent modules.

Each module performs a specific function.

For a Smart Inventory Management System, modules may include:

- Product Management
- Stock Management
- Supplier Management
- Sales Management
- Reporting
- Notification

Each module has a specific responsibility and can communicate with other modules through well-defined interfaces.

For example:

Product Module → Stock Module → Reporting Module

This makes the entire system easier to understand and manage.

(b) Procedural Programming vs Object-Oriented Programming

Feature	Procedural Programming	Object-Oriented Programming
Main Focus	Functions and procedures	Objects and classes
Data	Often separate from functions	Combined with methods
Approach	Top-down	Bottom-up
Reusability	Limited	High
Data Security	Less data protection	Encapsulation provides protection
Maintenance	Difficult for large systems	Easier to maintain
Real-World Modeling	Less natural	Closely represents real-world entities
Example	addProduct() function	Product class with addProduct() method

Procedural Approach Example

addProduct()

updateStock()

generateReport()

The system mainly focuses on functions.

Object-Oriented Approach Example

Product

- productName

- price

- quantity

+ addProduct()

+ updateStock()

The data and related operations are combined into objects.

(c) Benefits of Modularity

1. Scalability

New modules can be added without changing the entire system.

For example, a new AI Demand Prediction Module can be added to the inventory system.

2. Maintainability

Each module can be modified independently.

If the reporting module has an error, developers can fix it without changing the product management module.

3. Reusability

Modules can be reused in other systems.

For example, a Payment Module can be reused in:

- Inventory systems
- E-commerce systems
- Food delivery systems

4. Easier Testing

Each module can be tested separately before integration.

5. Reduced Complexity

A large system is divided into smaller, understandable parts.

6. Parallel Development

Different development teams can work on different modules simultaneously.

Therefore, modular software design improves scalability, maintainability, reusability, testing, and overall software quality.

Conclusion

Object-oriented concepts, design principles, SDLC models, UML diagrams, and modular design provide a strong foundation for developing reliable and maintainable software systems. By applying these concepts to real-world systems such as banking, healthcare, agriculture, transport, food delivery, and inventory management, software developers can create systems that are reusable, flexible, scalable, and easier to maintain.